

System Design & Implementation Plan
Project: Local Agent

Collaborative Design

September 29, 2025

Contents

1	Overview	2
2	Core Architecture	2
2.1	Cloud Services	2
2.2	Local Agent	2
3	Communication Protocol	2
4	Key Components & Technology Stack	2

1 Overview

The primary goal of this project is to create a secure, client-side application (the "Local Agent") that runs on the user's device. This agent will receive instructions from a cloud-based AI to execute commands, manage files, and interact with external APIs using locally stored, secure credentials. This document outlines the design, architecture, and implementation roadmap.

2 Core Architecture

The system is based on a "Cloud Thinking, Local Acting" model, which separates the resource-intensive reasoning from the secure, lightweight execution.

2.1 Cloud Services

The cloud-based AI backend is responsible for all complex tasks, including:

- Understanding user intent.
- Performing large-scale analysis and searches.
- Generating plans and source code.
- Formulating structured instructions for the Local Agent.

2.2 Local Agent

The Local Agent is a lightweight, secure executor that runs on the user's end device. Its sole responsibilities are:

- Receiving instructions from the cloud.
- Parsing and validating instructions.
- Executing actions on the local file system.
- Interacting with the OS's secure credential store.
- Making authenticated API calls to external services.

3 Communication Protocol

The cloud will send instructions to the agent via a standardized JSON format. This allows for clear, structured, and machine-readable commands. An example instruction is:

```
{
  "action": "create_file",
  "path": "./src/main.cs",
  "content": "public class Program { ... }"
}
```

4 Key Components & Technology Stack

Language C# and the .NET framework are chosen for their robustness, cross-platform capabilities, and strong support for secure application development.

Instruction Parser A module to deserialize JSON instructions into executable C# objects. Libraries like `System.Text.Json` will be used.

Action Executor A core module containing the logic to perform all defined actions (e.g., file I/O, command execution).

Secure Credential Manager A module that interacts with the operating system's native secure storage (e.g., Windows Credential Manager or macOS Keychain). The .NET `ProtectedData` class can also be used as a foundational layer.